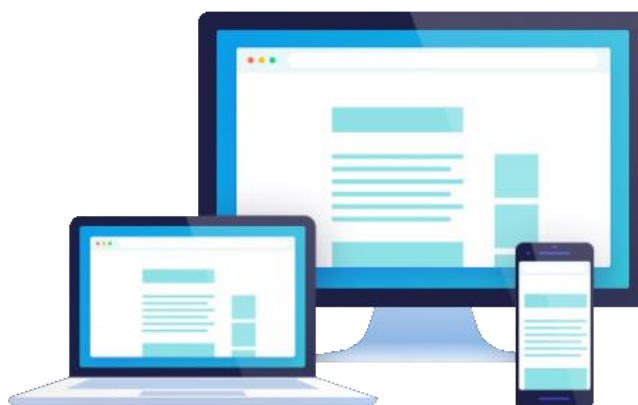


Application de la Démarche Scientifique

Un site internet plus réactif !

15/04/2022



Etudiant : GASTON Clément

Formation : A4 Informatique - CESI

Enseignant formateur : MASSRI Wassim

Mode de diffusion :

Restreinte ☐

Libre ☒

Remerciements

Je tiens à remercier toutes les personnes qui ont contribué au succès de mon stage et qui m'ont aidé lors de la rédaction de ce rapport.

Tout d'abord, j'adresse mes remerciements au CESI qui m'a permis de participer à un projet tel que celui-ci tout en étant encadré et guidé.

Je tiens à remercier également mon tuteur MASRI Wassim, pour son écoute et ces explications qui m'ont permis de comprendre et réussir au mieux cette démarche scientifique.

Résumé en Français

Le projet vient s'inscrire dans une démarche de recherche scientifique. Il a pour but de comprendre l'utilité du temps de chargement d'une page internet. Lors de cette étude, nous nous intéressons aux différentes optimisations WEB existant dans le monde avant de le tester une à une. Le but de ces expériences est de calculer le gain de chacune pour en trouver leur utilité ou non.

L'objectif aux travers de ces expériences et de savoir si une optimisation impact ou non le temps de chargement d'un site internet. Pour réussir cette étude, nous devons trouver au moins une optimisation qui améliore ce temps et dont il est possible d'en tirer une règle générale d'optimisation (par exemple : En moyenne, l'optimisation X permet de gagner Y% de temps ou permet de réduire de Z% la taille d'un fichier).

Pour chaque optimisation, une expérience spécifique y est attribuée. Pour les optimisations de taille de fichier, les différents fichiers seront comparés sur leurs tailles. Pour les optimisations permettant un affichage plus rapidement, nous nous intéresserons aux temps d'affichage de la page internet. Et pour finir, pour les optimisations sur les temps de chargement, nous nous concentrerons sur le temps moyen gagnés. Chaque expérience est reproductible (lien en annexe) et ces expériences sont réalisées plusieurs fois afin d'en augmenter la précision des résultats.

L'étude est réussie car, pour 5 des 8 optimisations, elles améliorent le temps de chargement d'une page et il est possible d'en tirer une règle d'optimisation générale.

Résumé en anglais

The project is part of a scientific research approach. It aims to understand the usefulness of the loading time of a web page. During this study, we are interested in the different WEB optimizations existing in the world before testing it one by one. The purpose of these experiments is to calculate the gain of each to find their usefulness or not.

The objective through these experiences is to know whether or not an optimization impacts the loading time of a website. To succeed in this study, we must find at least one optimization that improves this time and from which it is possible to derive a general optimization rule (for example: On average, optimization X saves Y% of time or reduces the size of a file by Z%).

For each optimization, a specific experience is attributed to it. For file size optimizations, different files will be compared on their sizes. For optimizations allowing a faster display, we will focus on the display times of the web page. And finally, for optimizations on loading times, we will focus on the average time saved. Each experiment is reproducible ([link in appendix](#)) and these experiments are carried out several times in order to increase the precision of the results.

The study is successful because, for 5 of the 8 optimizations, they improve the loading time of a page and it is possible to derive a general optimization rule.

Table des matières

Remerciements.....	3
Résumé en Français	4
Résumé en anglais	5
Table des figures / Citations	7
Introduction	8
1. Analyse de l'existant	13
1.1. Minification des fichiers JavaScript et CSS	13
1.2. Exclure les composants inutilisés des bibliothèques .js	14
1.3. Chargement asynchrone et différé pour les fichiers CSS et JavaScript.	15
1.4. Visibilité de contenu.....	15
1.5. Optimisation des images	16
1.6. Chargement des images "lazy"	16
1.7. Décodage asynchrone des images.....	17
1.8. L'attribut "srcset"	17
2. Etat de l'art.....	18
2.1. L'URL est résolue.....	18
2.2. La requête est envoyée	19
2.3. La réponse est analysée.....	20
2.4. La page est affichée.....	21
2.4.1. Fichier CSS	21
2.4.2. Fichier JavaScript	21
3. Expériences	23
3.1. Minification des fichiers JavaScript et CSS	23
3.2. Exclure les composants inutilisés des bibliothèques .js	24
3.3. Chargement asynchrone et différé pour les fichiers CSS et JavaScript.	24
3.4. Visibilité de contenu.....	25
3.5. Optimisation des images	25
3.6. Chargement des images "lazy"	26
3.7. Décodage asynchrone des images.....	26
3.8. L'attribut "srcset"	26
Conclusion.....	27
Bibliographie	29
Glossaire	31
Annexes.....	32

Table des figures / Citations

Figure 1: Taux de rebond en fonction du temps de chargement (en secondes)	9
Figure 2: Statistiques sur les sites web sur appareil mobiles	10
Figure 3: Statistiques sur les pertes encourues par un mauvais temps de chargement.....	11
Figure 4: Analyse de mauvaises optimisations sur 150 000 sites web.....	12
Figure 5: Explication visuelle d'une minification de fichier	13
Figure 6: Exemple de code d'une minification de fichier JavaScript.....	14
Figure 7: Exemple d'exclusion de composant d'une librairie JavaScript.....	14
Figure 8: Comparaison visuelle entre un tâche synchrone et asynchrone.....	15
Figure 9: Exemple d'utilisation de l'attribut srcset sur un code HTML	17
Figure 10: Explication du processus de chargement d'un site internet	18
Figure 11: Requête d'accès à un site	19
Figure 12: Réponse d'un site internet	20
Figure 13: Réponse sous format "text/html"	20
Figure 14: Exemple de code HTML.....	21
Figure 15: Minification des fichiers JavaScript (Boite à moustache).....	23
Figure 16: Minification des fichiers CSS (Boite à moustache).....	23
Figure 17: Courbe de tendance des fichiers JavaScript.....	23
Figure 18: Courbe de tendance des fichiers CSS	23
Figure 19: Comparaison du temps de téléchargements de fichiers JavaScript et CSS de façon asynchrone et synchrone.....	24
Figure 20: Comparaison du pourcentage de temps gagnés si l'on utilise un chargement asynchrone.	24
Figure 21: Chargement d'image SANS visibilité de contenu.....	25
Figure 22: Chargement d'image AVEC visibilité de contenu.....	25
Figure 23: Différence de taille d'une même image en fonction de son format.....	25
Figure 24: Pourcentage de place gagné par optimisation	26

Introduction

La vitesse de chargement des pages Internet est la première chose que vous remarquez lorsque vous visitez un site Web, mais vous ne le remarquerez que si le site Web est trop lent. La vitesse des pages est une partie importante de l'expérience utilisateur, il n'est donc pas étonnant qu'elle soit officiellement devenue un facteur de classement pour les recherches mobiles et qu'elle en soit un pour les ordinateurs fixes depuis des lustres.

De nombreuses études sur la corrélation entre la vitesse de chargement des pages et les taux de rebond ont abouti à des conclusions plutôt alarmantes : chaque milliseconde compte. En tant que développeur WEB, se renseigner sur les optimisations est primordial. Cependant, il n'existe aucune ressource listant les optimisations **et** leurs gains de temps. Il devient alors évident qu'une personne, possédant un site Internet, ne comprend pas l'intérêt de l'optimiser. D'autant plus que ce processus prend du temps et coûte de l'argent. Face à ce constant, la problématique de cette étude m'est apparue plus qu'évidente : « L'optimisation d'un site Internet, un processus réellement impactant ? ». Pour comprendre au mieux l'importance du chargement d'une page Internet, nous allons nous plonger plus en détails sur le sujet.

Avant de commencer, couvrons quelques définitions de base :

- Vitesse de la page WEB : La vitesse de la page fait référence à la mesure du temps nécessaire au chargement du contenu de votre page. Il peut être mesuré en "temps de chargement de la page" ou en "temps jusqu'au premier octet".
- Temps de chargement de la page : Le temps de chargement de la page est le temps qu'il faut pour afficher tout le contenu d'une page Web spécifique - à partir du moment où vous cliquez sur un lien de page jusqu'au moment où il est entièrement chargé dans votre navigateur.
- Temps jusqu'au premier octet : Le temps jusqu'au premier octet est une mesure de la réactivité de votre serveur Web. Il représente le temps nécessaire à votre navigateur Web pour recevoir le premier octet d'informations de votre serveur.
- Temps de rendu : Le temps de rendu est le temps nécessaire au serveur pour télécharger entièrement la page où les visiteurs peuvent utiliser et interagir avec le contenu de la page.
- Taux de rebond : Le taux de rebond est un indicateur marketing qui mesure le pourcentage d'internautes qui sont entrés sur une page Web et qui ont quitté le site après, sans consulter d'autres pages. Ils n'ont donc vu qu'une seule page du site. Pour calculer ce dernier, il suffit de faire le rapport entre ce type de visiteur et le nombre total de visiteurs.
- Taux de conversion : Représente la proportion d'achats sur un site web donné par rapport au nombre de visiteurs uniques ayant parcouru le site.

La meilleure pratique consiste à charger votre site Web en moins de deux à trois secondes, selon John Mueller, analyste principal des tendances pour les webmasters chez Google. Idéalement, un site Web devrait se charger le plus rapidement possible. Si vous possédez un site Web de commerce en ligne, vous devez viser un temps de chargement de moins de deux secondes où vous risquez de perdre des clients potentiels.

Par exemple, les pages qui se chargent en moins de deux secondes ont un taux de rebond moyen de 9%, tandis que les pages qui se chargent en cinq secondes voient leur taux de rebond grimper à 38%. Même une différence d'une seconde dans le temps de chargement peut avoir un impact considérable sur les performances :

TAUX DE REBOND

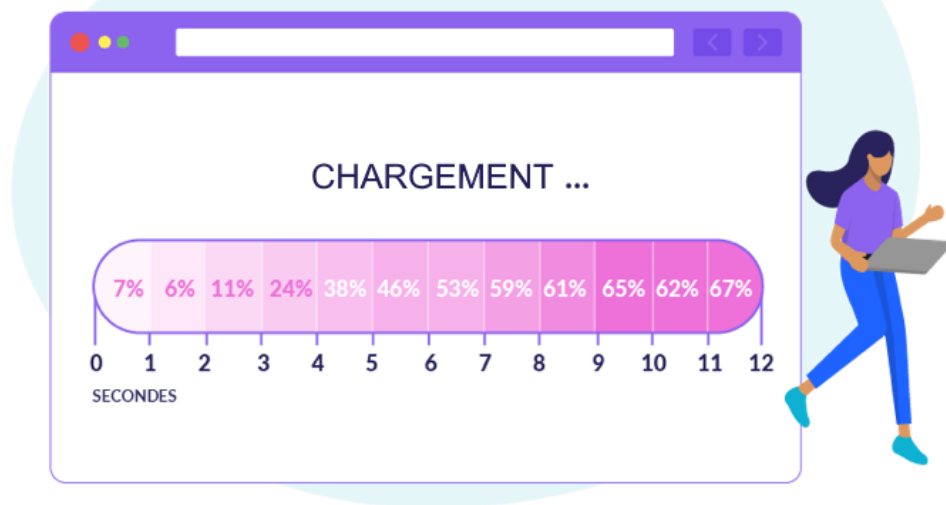


Figure 1: Taux de rebond en fonction du temps de chargement (en secondes)

Un temps de chargement de page d'une seconde permet aux utilisateurs de se sentir sans stress et en contrôle. Mais après 10 secondes, leur attention est à peine retenue et il est peu probable qu'ils visitent à nouveau la page Web. Au fur et à mesure que les secondes augmentent, le taux de rebond augmente également. Ce qui diminue, c'est la patience de l'utilisateur ! Idéalement un site Web doit se charger en trois ou en deux secondes s'il s'agit d'un site de commerce électronique. La marque des deux à trois secondes est le tournant où les taux de rebond montent en flèche. En fait, 40% des consommateurs n'attendront pas plus de trois secondes avant d'abandonner un site.

Même les grands noms ne sont pas à l'abri des effets de la vitesse du site Web. La BBC perd 10 % d'utilisateurs supplémentaires pour chaque seconde supplémentaire nécessaire au chargement de son site. Et quand Yahoo! a réduit le temps de chargement de sa page de seulement 400 millisecondes, le trafic a augmenté de 9 %.

De nos jours, les ordinateurs sont de plus en plus délaissés par les utilisateurs pour naviguer sur Internet, privilégiant de plus en plus le téléphone mobile. Le temps que les gens passent sur leurs appareils mobiles augmente d'année en année. Le mobile devenant le moyen le plus populaire de naviguer en ligne, il est essentiel pour les entreprises de garder à l'esprit la vitesse de leur page mobile car une expérience utilisateur rapide est impérative.

En 2017, [Google a effectué une analyse de 900 000 pages d'accueil](#) dans plusieurs pays et secteurs industriels pour comprendre l'état de la vitesse mobile. Leur consensus : la majorité des sites mobiles sont trop lents. Ils ont constaté que le temps moyen de chargement d'une page de destination mobile était de 22 secondes, bien que plus de 50 % des visiteurs quittent une page qui prend plus de trois secondes à se charger.

En 2019, [Backlinko a mené une étude](#) analysant cinq millions de pages sur ordinateurs et mobiles. Ils ont constaté que le temps moyen nécessaire pour charger complètement une page Web était de **10,3 secondes sur ordinateur** et de **27,3 secondes sur mobile**.

Il est prudent de dire que la plupart des pages d'accueil sur mobiles et ordinateurs ont du travail à faire dans le domaine de la vitesse de chargement. Pour bien comprendre l'état du temps de chargement des pages mobiles par rapport aux ordinateurs de bureau, nous couvrons des statistiques supplémentaires ci-dessous :

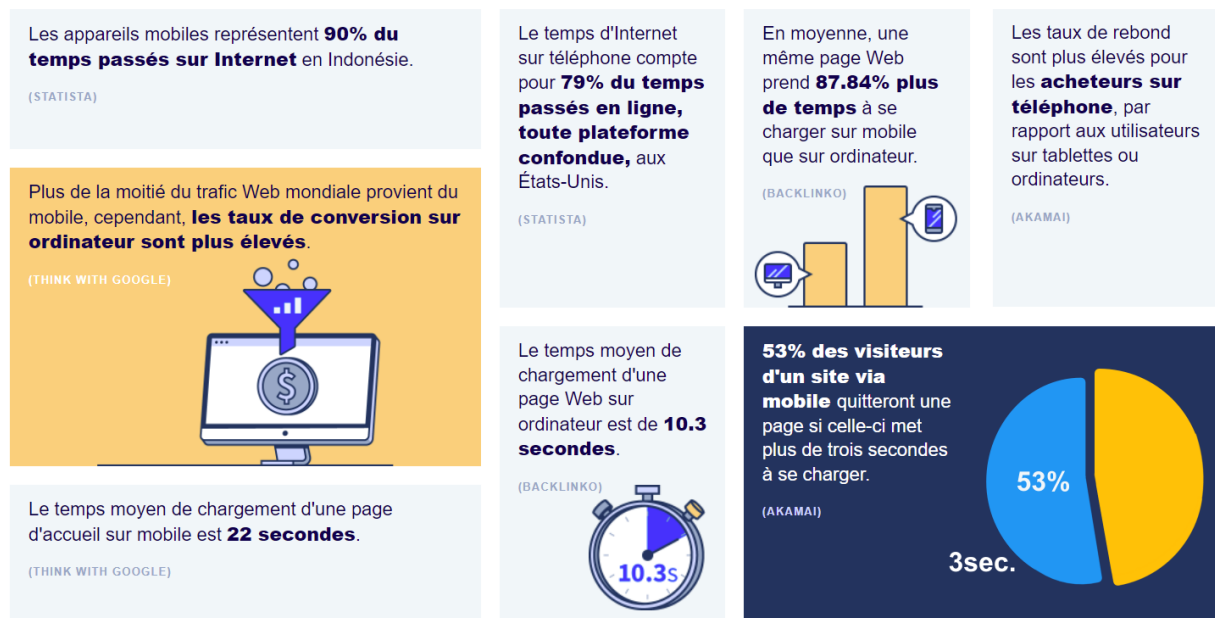


Figure 2: Statistiques sur les sites web sur appareil mobiles

Maintenant que nous avons vu que la vitesse des pages affecte l'expérience utilisateur, nous ne savons pas à quel point cela peut être préjudiciable. Le temps de chargement d'un site Web a un impact direct sur le comportement des consommateurs. Plus le temps de chargement de votre page est long, plus le taux de rebond sera élevé. Si le site est exploité par une entreprise en ligne, un temps de chargement lent peut entraîner un taux de rebond plus élevé, une perte générale de trafic et une perte importante de conversions.

Aujourd'hui, les consommateurs veulent des informations rapides et ils n'ont pas la patience d'attendre. La durée d'attention moyenne de la génération Z est de huit secondes, tandis que la durée d'attention moyenne de la génération précédente n'est supérieure que de quelques secondes. Attirer l'attention du consommateur et la conserver est un exploit en soi, surtout avec la réduction de la durée d'attention. Plutôt que d'attendre qu'une page se charge, les consommateurs d'aujourd'hui préfèrent rechercher une nouvelle page.

La vitesse du site Web peut avoir un impact sur tout, des ventes aux performances des moteurs de recherche, ce qui signifie que les sites de commerce électronique paieront littéralement le prix des retards. Si vous possédez un site de commerce en ligne qui rapporte 100 000 \$ par jour, un retard d'une seconde de page pourrait vous coûter 2,5 millions de dollars par an en ventes perdues. Tout site Web lent, de commerce électronique ou autre, verra une augmentation du taux de rebond et une diminution du temps passé sur le site. Même un simple délai d'une seconde sera...

- Réduit les pages vues de 11 %
- Diminue la satisfaction client de 16 %
- Réduit les conversions de 7 %

Si vous essayez de faire des ventes en ligne, les pertes de revenus résultant des vitesses lentes seront assez destructrices. D'autant plus lorsque l'on sait que :

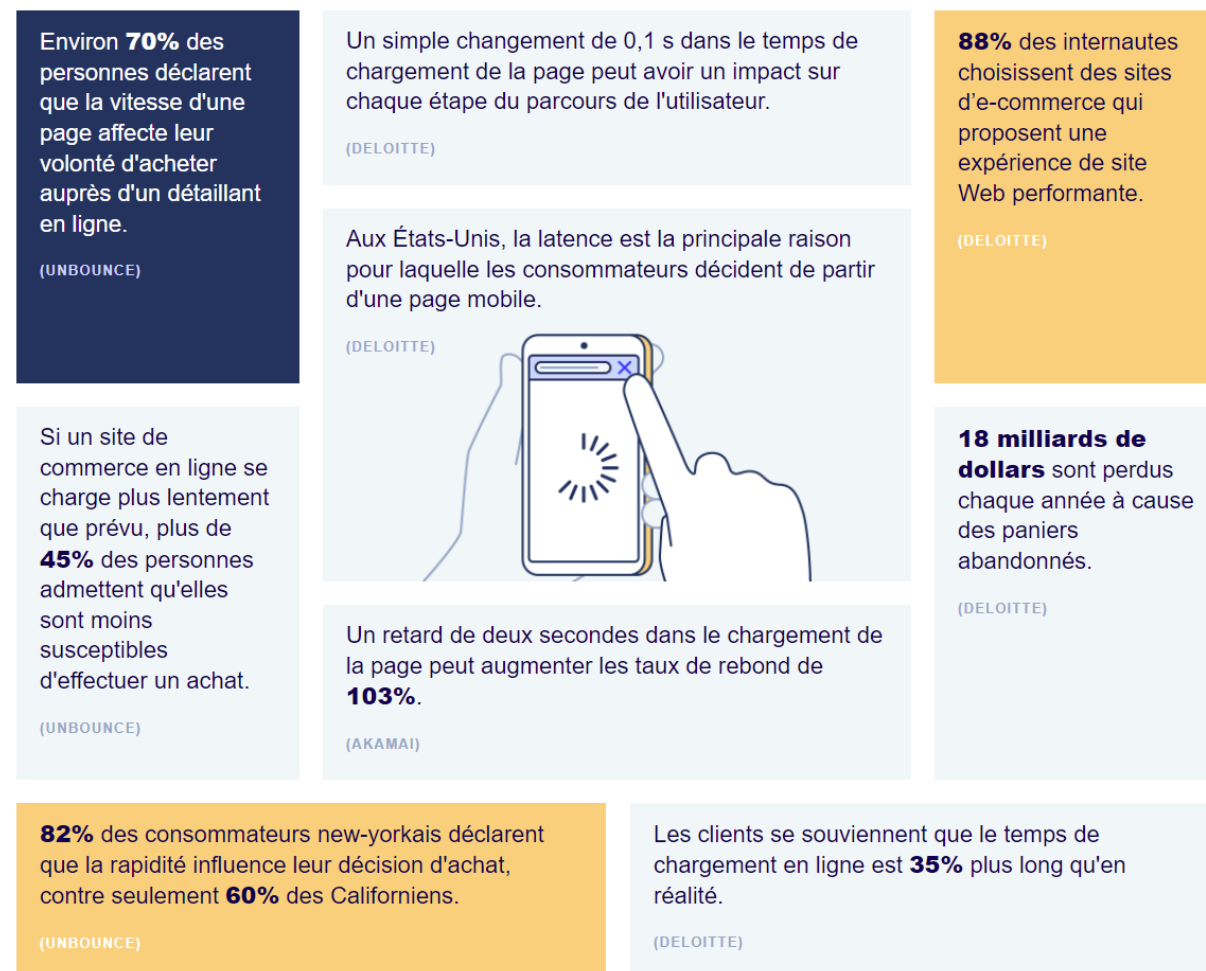


Figure 3: Statistiques sur les pertes encourues par un mauvais temps de chargement

Mais d'une autre côté, les gains apportés par une amélioration du temps de chargement sont également surprenants. Les bénéfices pour une augmentant la vitesse du site de seulement 0,1 seconde sont :

- Taux de conversions augmentés de 8.4%
- Valeur moyenne des commandes augmentés de 9.2%
- Pages de marques de luxe vues par session augmentés de 8.6%

La vitesse a également un impact sur la façon dont votre entreprise se comportera face à la concurrence. 79% des acheteurs en ligne qui ont une expérience insatisfaisante sont moins susceptibles d'acheter à nouveau sur le même site, tandis que 64 % achèteraient simplement dans une autre boutique en ligne.

Les statistiques vues précédemment nous laisse penser qu'il est inimaginable qu'un site Web puisse ne pas être optimisé. Les bénéfices en sont bien trop grands ! Pour vérifier cela, nous allons nous pencher sur une étude réalisée sur 150 000 sites Web choisi aléatoirement (versions bureau et mobiles) qui évalue leurs performances. L'étude est basée sur le rapport de performance de [SEMrush Site Audit](#), qui fournit des statistiques pratiques sur les neuf problèmes les plus courants qui affectent les performances du site Web :



Figure 4: Analyse de mauvaises optimisations sur 150 000 sites web

82 % des sites Web analysés ont des problèmes affectant considérablement leurs performances. Cela fait des problèmes liés aux performances l'un des plus courants sur les sites Web. De plus, 44 % des sites Web analysés présentaient au moins un problème critique (selon une classification arbitraire de gravité des problèmes d'audit de site).

Sur une note positive, nous pouvons constater que la majorité des sites Web rencontrant des problèmes de performances peuvent être améliorés en utilisant des techniques d'optimisation de page simples.

Nous avons vu qu'il est plus qu'important d'optimiser un site Web mais que, dans la réalité, beaucoup de personnes ne le font pas. En me baladant sur internet, je me suis retrouvé assez régulièrement face à des pages très lentes et je me suis reconnue dans les statistiques précédemment citées. Ces statistiques m'ont aussi fait prendre conscience de l'importance du temps de chargement des sites. En tant que développeur Web dans le monde professionnel, la problématique de cette étude m'est apparue claire : « L'optimisation d'un site Internet, un processus réellement impactant ? ».

Cette étude va permettre de comprendre, à mon niveau, si mon rôle de développeur est impactant. C'est-à-dire, si les différentes optimisations dans le code influent de façon significative le temps de chargement et s'il est possible d'en calculer le gain. Une optimisation sera considérée comme valide dès lors qu'elle améliore le temps de chargement par rapport à la solution initiale dite « non optimisée » et qu'il est possible d'en tirer une règle générale de gain de temps. L'objectif étant de dresser un tableau d'importance des optimisations. Ce tableau comportera le niveau de difficultés de mise en place de la solution et son gain en temps. Il aura pour but d'être une « checklist » d'optimisation à réaliser et à prioriser pour obtenir un maximum de résultat en peu de temps / ressources pour n'importe qui.

La première étape est d'effectuer une analyse de l'existant puis d'établir un état de l'art. Comme aucune ressource n'existe sur le gain de temps amené par les différentes optimisations, cette première partie servira de répertoire. Elle regroupera toutes les solutions pour améliorer un site Internet. Dans un second temps nous testerons et comparerons les solutions listées dans la partie précédente. Le test de ces optimisations se fera une par une et sa mise en place sera détaillée. Pour chaque optimisation, le gain de temps ne sera pas mesuré en secondes mais en pourcentage de temps gagnés ou en octets sauvés. En effet, il n'est pas pertinent de les mesurer en secondes car énormément de facteurs influent.

Cette étude ne traitera pas :

- De la configuration physique et/ou réseau côté client / serveur
- Des optimisations physiques et/ou réseau côté client / serveur
- Des optimisations de connexion internet

Nous ne nous concentrerons qu'autour des optimisations côté images, CSS et JavaScript.

1. Analyse de l'existant

Comme dit précédemment, aucunes ressources n'existent sur le gain de temps amené par les différentes optimisations. L'analyse de l'existant va être composé de toutes les solutions existantes pour améliorer un site Internet. Toutes ces solutions seront, par la suite, traitées dans la partie des expériences pour être tester afin d'en calculer le temps gagnés lors du chargement de la page.

1.1. Minification des fichiers JavaScript et CSS

L'optimisation de la façon dont les fichiers se chargent peut aider à améliorer la vitesse de chargement des pages. De même, il est également possible de réduire le code CSS, JavaScript et HTML. L'optimisation de la vitesse ne consiste pas (seulement) à réduire la taille de la page. La réduction du nombre de requêtes client-serveur lors de la livraison du contenu du site Web aux navigateurs Web fait partie intégrante de l'optimisation de la vitesse du site Web. Les webmasters peuvent il n'y parvenir en ne faisant pas trop de requête CSS, HTML et JavaScript. Bien que le nombre de requêtes ne soit plus aussi important qu'avant grâce aux améliorations de HTTP/2. La solution est alors d'optimiser, minifier et écraser toutes les lignes de code inutiles et compressibles.

La minification est particulièrement importante pour le JavaScript en ligne et les fichiers externes non mis en cache. Google recommande de minimiser tous les fichiers JavaScript de plus de 4096 octets et de réduire d'au moins 25 octets pour produire une différence notable par rapport au temps de chargement de la page.

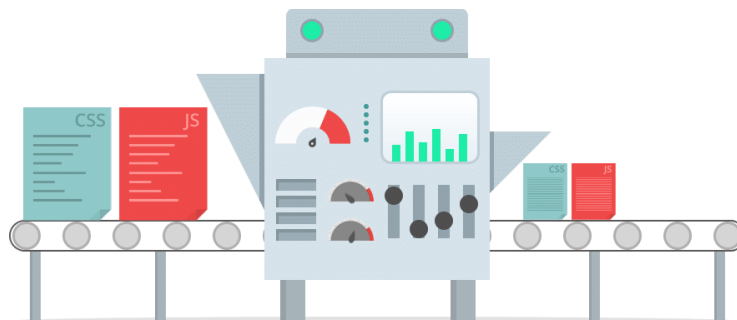


Figure 5: Explication visuelle d'une minification de fichier

Une approche rigoureuse de la conception d'un site Web optimisé pour la vitesse doit intégrer une forte intention d'éliminer les octets inutiles du code. L'utilisation de tout l'espace de codage disponible, la suppression des espaces supplémentaires, des indentations et des espaces de ligne tout en maintenant la lisibilité du code réduit la taille globale des fichiers principaux d'un site Web. Et pour les sites Web déjà développés sans cette stratégie en perspective, la combinaison de plusieurs requêtes de serveur (pour HTML, JavaScript et CSS) en une seule réduit efficacement les temps de chargement des pages.

Cependant, une minification excessive sur un fichier HTML conduit potentiellement à une perte de compréhension du code du site Web. Les agents utilisateurs prennent alors des cycles de mémoire et une puissance CPU excessifs pour « deviner » les ressources manquantes nécessaires pour analyser les fichiers HTML. La surveillance des modifications des performances de chargement des pages en réponse à la mise en œuvre de chaque processus de minification garantit que seuls le code et les espaces inutiles sont supprimés.

La minimisation de CSS, JavaScript et HTML partage des avantages communs : réduction de la latence du réseau, moins de requêtes HTML, compression améliorée, téléchargement et exécution plus rapides du navigateur, augmentant finalement la vitesse des pages et obtenant des scores plus élevés sur les outils de mesure de la vitesse du site Web.

Comment fonctionne l'optimisation ? Voici un exemple :

```
function test(node) {  
  var parent = node.parentNode;  
  if (0) {  
    alert( "Hello from the other side" );  
  } else {  
    alert( "We love Upwork" );  
  }  
  return;  
  alert( 1 );  
}  
➡ function test(e){e.parentNode;alert("We Love Upwork");}
```

Figure 6: Exemple de code d'une minification de fichier JavaScript

Qu'est-ce que l'optimisation a accompli exactement ?

- La variable parent n'aura jamais besoin d'être utilisée, elle est donc supprimée ;
- False if() {...} est supprimé en tant que "code mort" ; true else ne laisse qu'une seule possibilité.
- Le retour est supprimé ; c'est aussi du code mort.

1.2. Exclure les composants inutilisés des bibliothèques .js

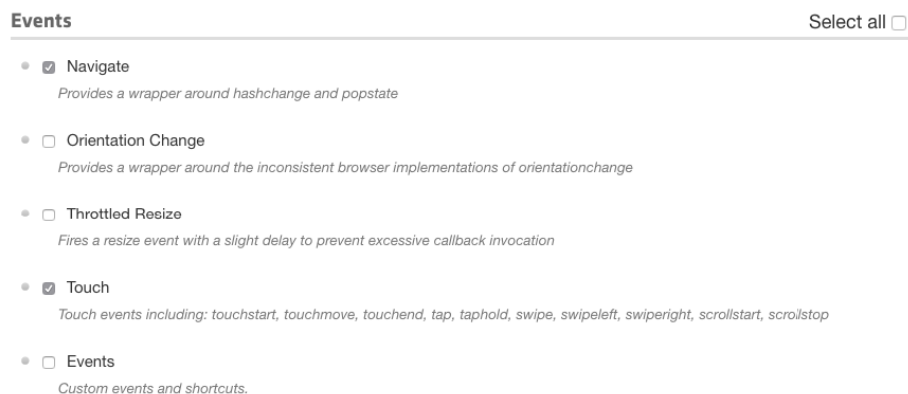


Figure 7: Exemple d'exclusion de composant d'une librairie JavaScript

La plupart des développeurs utilisent des bibliothèques telles que jQuery UI ou jQuery Mobile telles quelles. Cela signifie que le code inclut tous les composants possibles de chaque bibliothèque, alors que vous n'en aurez peut-être besoin que de deux ou trois. Une situation similaire se produit également avec d'autres bibliothèques JavaScript. S'il y a possibilité de gérer les composants qui seront inclus dans le package de bibliothèque, il est vivement conseillé de le faire. Le site Web se chargera beaucoup plus rapidement et les visiteurs auront une meilleure expérience.

1.3. Chargement asynchrone et différé pour les fichiers CSS et JavaScript.

Un site est composé de fichiers CSS et JavaScript. Ces scripts peuvent se charger de manière synchrone ou asynchrone. Synchrone signifie que les fichiers se chargent un par un, dans l'ordre dans lequel ils apparaissent sur votre page Web. Avec cette méthode, lorsque le navigateur rencontre un script, il arrête de charger d'autres éléments sur la page jusqu'à ce que ce fichier soit entièrement chargé en premier. À l'inverse, le chargement asynchrone permet à plusieurs fichiers de se charger en même temps, ce qui peut accélérer les performances de la page. Cette configuration implique l'élimination des ressources bloquant le rendu.

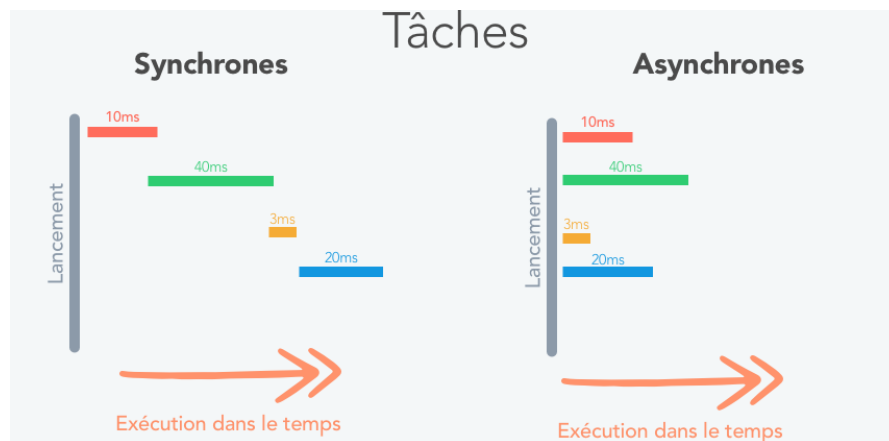


Figure 8: Comparaison visuelle entre un tâche synchrone et asynchrone

1.4. Visibilité de contenu

La propriété CSS « content-visibility » garantit que vous obtenez les gains de performances les plus importants que le navigateur peut fournir avec un minimum d'effort de votre part en tant que développeur. Elle accepte plusieurs valeurs, mais « auto » est celle qui fournit des améliorations de performances immédiates. Un élément qui possède cette valeur gagne en mise en page, en style et en rendu. Si l'élément est hors écran (et n'est pas autrement pertinent pour l'utilisateur - les éléments pertinents seraient ceux qui ont le focus ou la sélection dans leur sous-arborescence), il gagne en taille de calcul (et il arrête de peindre et de tester son contenu).

Qu'est-ce que ça veut dire ? En bref, si l'élément est hors écran, ses descendants ne sont pas rendus. Le navigateur détermine la taille de l'élément sans tenir compte de son contenu, et il s'arrête là. La plupart des rendus, tels que le style et la disposition de la sous-arborescence de l'élément sont ignorés. Lorsque l'élément s'approche de la fenêtre d'affichage, le navigateur n'ajoute plus la limitation de taille et commence à rendre et à tester le contenu de l'élément. Cela permet au travail de rendu d'être effectué juste à temps pour être vu par l'utilisateur.

1.5. Optimisation des images

En règle générale, les fichiers volumineux prennent plus de temps à télécharger que les fichiers plus petits. Le temps de téléchargement de la page Web, également appelé temps de chargement de la page, dépend de la taille totale du contenu téléchargé des serveurs d'hébergement vers le navigateur demandeur. Les images volumineuses de haute qualité sont les plus grands contributeurs à la taille de la page Web, dégradant la vitesse de la page et agitant les visiteurs qui attendent avec impatience le chargement de la page Web.

Selon [HTTP Archive](#), en décembre 2017, les images représentaient en moyenne 66% du poids total d'une page Web. Les meilleures pratiques d'optimisation d'image suivantes contribuent grandement à réduire l'impact négatif des images sur la vitesse du site Web :

- Sélection du format : Utilisez des fichiers JPG ou WEBP lorsque la qualité est une priorité élevée et qu'il n'est pas nécessaire de modifier l'image avant de la télécharger. Les JPG peuvent nécessiter un traitement et des modifications limités avant que la qualité de l'image ne se dégrade fortement. Pour les images avec des icônes, des logos, des illustrations, des signes et du texte, utilisez le format PNG. N'utilisez les GIF que pour les petites images ou les images simples et évitez les BMP ou les TIFF.
- Dimensionnement approprié : Économisez de précieux octets et faites correspondre les dimensions (largeur) de votre modèle de page Web. Utilisez les capacités de redimensionnement du navigateur pour rendre les images réactives en définissant des instructions de largeur fixe et de hauteur automatique.
- Compression : La compression d'image doit être un compromis judicieux entre la taille et la qualité de l'image. Pour les JPG, une compression de 60 à 70% produit un bon équilibre. Pour les écrans Retina, augmentez la taille de l'image (JPG) de 150 à 200%, compressez-la de 30 à 40% et réduisez-la à nouveau selon les dimensions requises.
- Moins d'images : Gardez le nombre d'images à un minimum absolu.

1.6. Chargement des images "lazy"

Avec l'attribut « loading » sur une balise IMG, les utilisateurs chargeront l'image lorsqu'elle est spécifiée. Le chargement différé (Lazy) est une stratégie pour identifier les ressources comme non bloquantes (non critiques) et les charger uniquement lorsque cela est nécessaire. C'est une bonne stratégie pour éviter d'encombrer les utilisateurs avec plusieurs requêtes, en chargeant les éléments critiques de la page Web avant les autres.

Parfois, les images ne sont que des décorations de la page Web, pas des aspects critiques, de sorte que les informations peuvent être chargées après le contenu principal. Dans ce cas, la valeur « lazy » est la meilleure option. Dans d'autres cas, les images peuvent être la partie la plus importante de l'application Web, nous devons donc les charger dès que possible. Dans ce cas, la valeur « eager » est la voie à suivre.

1.7. Décodage asynchrone des images

Les images doivent être rendues sur la page Web par les navigateurs. Le décodage est l'opération consistant à élaborer des octets d'informations pour colorer correctement les pixels. Bien sûr, cette opération coûte de nombreux cycles CPU, augmentant le temps de rendu total de la page Web. Pour améliorer les performances, nous pouvons spécifier l'image à décoder de manière asynchrone.

1.8. L'attribut "srcset"

Avec les jeux de sources, nous pouvons basculer très facilement entre différentes images pour différentes résolutions. Prenons pour exemple :

```

```

Figure 9: Exemple d'utilisation de l'attribut srcset sur un code HTML

Le "x" fait référence à la densité de pixels, la petite image (small) est la réplique exacte de la moyenne (medium) mais elle est réduite de 50 %, tandis que la moyenne (medium) est la réplique exacte de la grande mais réduite de 50 %. Il est également possible d'utiliser "w" au lieu de "x" en précisant, dans ce cas, la largeur exacte de l'écran en pixels. De cette façon, les appareils avec moins de pixels à rendre chargeront des images plus petites, ce qui se traduira par un rendu globalement plus rapide.

2. Etat de l'art

Comment un site internet fonctionne-t-il ? Quelles technologies sont impliquées et que devez-vous savoir ? Commençons par la façon la plus évidente d'utiliser Internet : vous visitez un site Web comme google.com. Au moment où vous entrez cette adresse dans votre navigateur et que vous appuyez sur ENTER, beaucoup de choses différentes se produisent :

- L'URL est résolue
- Une demande est envoyée au serveur du site Web
- La réponse du serveur est analysée
- La page est rendue et affichée

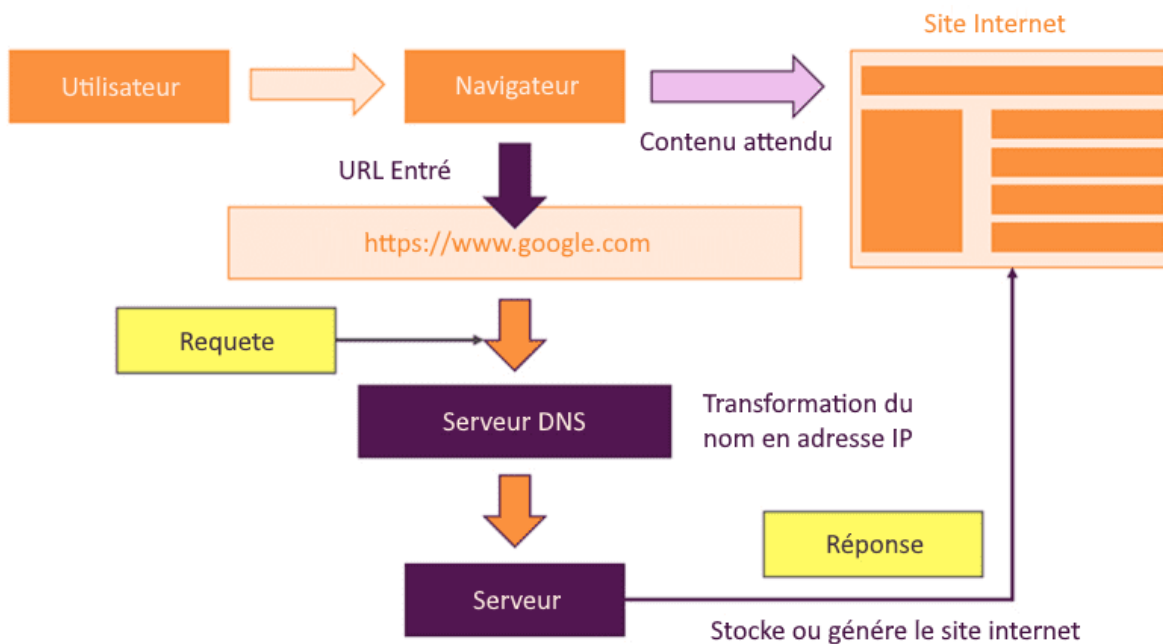


Figure 10: Explication du processus de chargement d'un site internet

En fait, chaque étape peut être divisée en plusieurs autres étapes, mais pour un bon aperçu de la façon dont tout cela fonctionne, c'est quelque chose que nous pouvons ignorer ici. Jetons un coup d'œil aux quatre étapes.

2.1. L'URL est résolue

Le code du site Web n'est évidemment pas stocké sur votre machine et doit donc être récupéré sur un autre ordinateur où il est stocké. Cet "autre ordinateur" est appelé un "serveur". Parce qu'il sert à quelque chose, dans notre cas, il sert le site Web.

Vous saisissez "google.com" (que l'on appelle "un domaine") mais en réalité, le serveur qui héberge le code source d'un site web, est identifié via des adresses IP (= Internet Protocol). Le navigateur envoie une "requête" (voir étape 2) au serveur avec l'adresse IP que vous avez saisie (indirectement - vous avez bien entendu saisi " google.com").

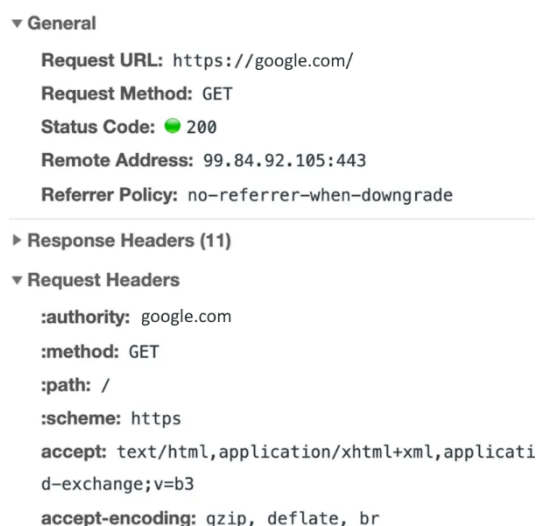
Une adresse IP ressemble généralement à ceci : 172.56.180.5 (bien qu'il existe également une forme plus "moderne" appelée IPv6 - mais ignorons cela pour l'instant). Comment le domaine "google.com" est-il traduit en adresse IP ? Il existe un type spécial de serveur sur Internet - pas seulement un mais

plusieurs serveurs de ce type. Un soi-disant "serveur de noms" ou "serveur DNS" (où DNS = "Domain Name System").

Le travail de ces serveurs DNS est de traduire les domaines en adresses IP. Vous pouvez imaginer ces serveurs comme d'énormes dictionnaires qui stockent des tables de traduction : Domaine => Adresse IP. Lorsque vous entrez "google.com", le navigateur récupère donc d'abord l'adresse IP d'un tel serveur DNS. Le navigateur connaît par cœur les adresses de ces serveurs de domaine, elles sont programmées dans le navigateur pour ainsi dire.

2.2. La requête est envoyée

Une fois l'adresse IP résolue, le navigateur continue et fait une demande au serveur avec cette adresse IP. "Une demande" n'est pas qu'un terme. C'est vraiment une chose technique qui se passe dans les coulisses. Le navigateur regroupe un tas d'informations (quelle est l'URL exacte ? Quel type de demande doit être faite ? Les métadonnées doivent-elles être jointes) et envoie ce paquet de données à l'adresse IP.



Les données sont envoyées via le "HyperText Transfer Protocol" (connu sous le nom de "HTTP") - un protocole standardisé qui définit à quoi une requête (et une réponse) doit ressembler, quelles données peuvent être incluses (et sous quelle forme) et comment la demande sera déposée. Vous pouvez en savoir plus sur HTTP [ici](#).

Étant donné que l'ensemble du processus et du format sont normalisés, il n'y a aucune hypothèse sur la façon dont cette demande doit être lue par le serveur.

Figure 11: Requête d'accès à un site

Le serveur traite alors la demande de manière appropriée et renvoie une "réponse". Encore une fois, une "réponse" est une chose technique et un peu similaire à une "demande". Vous pourriez dire que c'est essentiellement une "demande" dans la direction opposée. Comme une demande, une réponse peut contenir des données, des métadonnées, etc. Lorsque vous demandez une page comme [academind.com](#), la réponse contiendra le code requis pour afficher la page à l'écran.

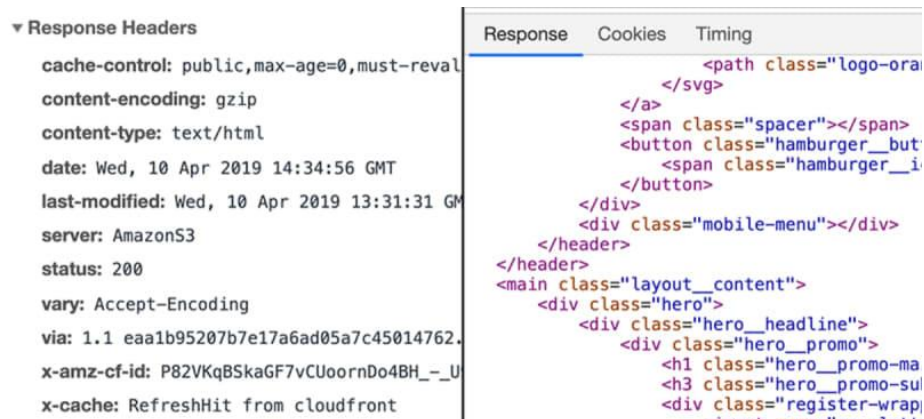


Figure 12: Réponse d'un site internet

Que se passe-t-il sur le serveur ? Cela est défini par les développeurs Web. Au final, une réponse doit être envoyée. Cette réponse ne doit pas nécessairement contenir "un site Web". Il peut contenir n'importe quelle donnée, y compris des fichiers ou des images.

Certains serveurs sont programmés pour générer dynamiquement des sites Web en fonction de la demande (par exemple, une page de profil contenant vos données personnelles), d'autres serveurs renvoient des pages HTML pré-générées (par exemple, une page d'actualités). Ou les deux sont effectués - pour différentes parties d'une page Web. Il existe également une troisième alternative : les sites Web qui sont pré-générés mais qui modifient leur apparence et leurs données dans le navigateur.

2.3. La réponse est analysée

Le navigateur reçoit la réponse envoyée par le serveur. Cela seul, n'affiche cependant rien à l'écran. Au lieu de cela, l'étape suivante consiste à ce que le navigateur analyse la réponse. Tout comme le serveur l'a fait avec la demande. Encore une fois, la normalisation imposée par HTTP aide bien sûr. Le navigateur vérifie les données et les métadonnées incluses dans la réponse. Et sur cette base, il décide quoi faire. Vous avez peut-être eu des cas où un PDF s'est ouvert dans votre navigateur. Cela s'est produit parce que la réponse a informé le navigateur que les données ne sont pas un site Web mais un document PDF à la place. Et le navigateur essaie de choisir le meilleur mécanisme de gestion pour tout type de données qu'il détecte.

Retour à notre scénario de site Web. Dans ce cas, la réponse contiendrait un élément spécifique de métadonnées, qui indique au navigateur que les données de réponse sont de type text/html.

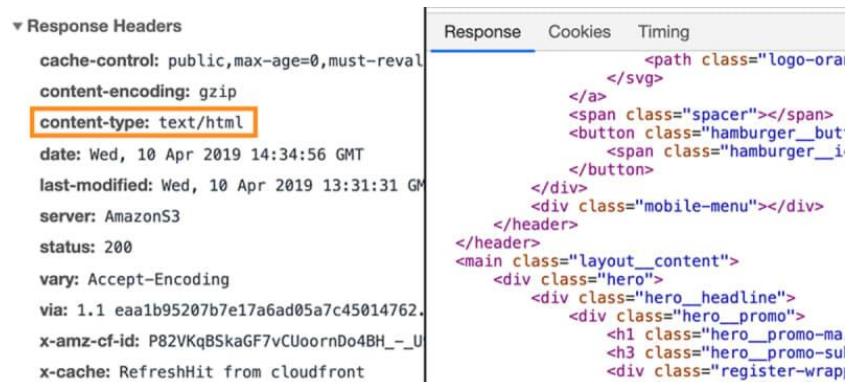


Figure 13: Réponse sous format "text/html"

Cela permet au navigateur d'analyser ensuite les données réelles attachées à la réponse sous forme de code HTML. HTML est le "langage de programmation" de base (techniquement, ce n'est pas un langage de programmation - vous ne pouvez pas écrire de logique avec) du Web. HTML signifie "Hyper Text Markup Language" et décrit la structure d'une page Web.

Le code ressemble à ceci :

```
<h1>Breaking News!</h1>  
<p>Websites work because browser understand HTML!</p>
```

Figure 14: Exemple de code HTML

<h1> et <p> sont appelés "balises HTML". Chaque balise HTML a une signification sémantique que le navigateur comprend, car le HTML est également standardisé. Par conséquent, il n'y a aucune supposition sur la signification d'une balise <h1>. Le navigateur sait comment analyser le code HTML et parcourt maintenant simplement l'intégralité des données de réponse (également appelées "le corps de la réponse") pour afficher le site Web.

2.4. La page est affichée

Comme mentionné, le navigateur parcourt les données HTML renvoyées par le serveur et construit un site Web basé sur cela. Bien qu'il soit important de savoir que le HTML n'inclut aucune instruction concernant ce à quoi le site devrait ressembler (c'est-à-dire comment il devrait être stylisé). Il définit uniquement la structure et indique au navigateur quel contenu est un titre, quel contenu est une image, quel contenu est un paragraphe, etc. Ceci est particulièrement important pour l'accessibilité - les lecteurs d'écran obtiennent toutes les informations utiles de la structure HTML. Un site WEB charge généralement deux autres types de fichiers : Les fichiers CSS et JavaScript.

2.4.1. Fichier CSS

CSS signifie feuilles de style en cascade. C'est le langage pour décrire la présentation des pages Web, y compris les couleurs, la mise en page et les polices, rendant ainsi nos pages Web présentables aux utilisateurs. CSS est conçu pour créer des feuilles de style pour le Web. Il est indépendant de HTML et peut être utilisé avec n'importe quel langage de balisage basé sur XML. Essayons maintenant de casser l'acronyme :

- Cascade : chute des styles
- Style : Ajouter des designs/Style nos balises HTML
- Fiches : Rédaction de notre style dans différents documents

2.4.2. Fichier JavaScript

JavaScript est un langage de script multiplateforme orienté objet utilisé pour rendre les pages Web interactives (par exemple, avoir des animations complexes, des boutons cliquables, des menus contextuels, etc.). Il existe également des versions côté serveur plus avancées de JavaScript telles que Node.js, qui vous permettent d'ajouter plus de fonctionnalités à un site Web que le téléchargement de fichiers (comme la collaboration en temps réel entre plusieurs ordinateurs). Dans un environnement hôte (par exemple, un navigateur Web), JavaScript peut être connecté aux objets de son environnement pour fournir un contrôle programmatique sur eux.

JavaScript contient une bibliothèque standard d'objets, tels que Array, Date et Math, et un ensemble de base d'éléments de langage tels que des opérateurs, des structures de contrôle et des instructions. Core JavaScript peut être étendu à diverses fins en le complétant avec des objets supplémentaires ; par exemple :

- JavaScript côté client étend le langage de base en fournissant des objets pour contrôler un navigateur et son modèle d'objet de document (DOM). Par exemple, les extensions côté client permettent à une application de placer des éléments sur un formulaire HTML et de répondre aux événements de l'utilisateur tels que les clics de souris, la saisie de formulaire et la navigation dans les pages.
- JavaScript côté serveur étend le langage de base en fournissant des objets pertinents pour l'exécution de JavaScript sur un serveur. Par exemple, les extensions côté serveur permettent à une application de communiquer avec une base de données, d'assurer la continuité des informations d'un appel à l'autre de l'application ou d'effectuer des manipulations de fichiers sur un serveur.

Cela signifie que dans le navigateur, JavaScript peut modifier l'apparence de la page Web (DOM). De même, Node.js JavaScript sur le serveur peut répondre aux demandes personnalisées à partir du code écrit dans le navigateur.

3. Expériences

Les différentes solutions qui vont être testées lors de cette étude sont l'ensemble des optimisations cités lors de l'analyse de l'existant. Pour chaque optimisation, une expérience sera mise en place. La mise en place sera détaillée et l'expérience sera réfléchi de façon à ce que les résultats soient le plus pertinent possible. Chacune de ces expériences peuvent être refaite à l'aide des fichiers en annexes. Une solution sera validée, si et seulement si, elle optimise le temps de chargement d'une page et qu'elle suit une règle définie. Par exemple, une optimisation dont il est impossible de prédire le résultat sera invalide même si elle améliore le temps de chargement d'une page.

3.1. Minification des fichiers JavaScript et CSS

La minification des fichiers JavaScript et CSS consiste à enlever tous les éléments non essentiels à la compréhension du code par la machine (suppression des espaces, des commentaires...). Pour tester cette solution, 50 fichiers CSS et 50 fichiers JavaScript seront soumis au test. Les fichiers choisis sont tirés de GitHub et respectent les conventions de codage. Pour un fichier ne respectant pas les normes de codages, les résultats peuvent être différents.

Pour chacun des fichiers, nous le téléchargeons et regardons sa taille en octets. Ensuite, nous faisons de même pour sa version minimisée. Pour chaque couple de fichier (version normale et minimisée), nous comparons les résultats pour en tirer un pourcentage de gain de taille. Dans un second temps, nous comparons les 50 différents résultats pour voir si une règle générale peut être trouvée. Pour cela, nous allons utiliser l'outil de la boîte à moustache qui va nous permettre de regarder s'il existe des valeurs extrêmes :



Figure 15: Minification des fichiers JavaScript (Boîte à moustache)

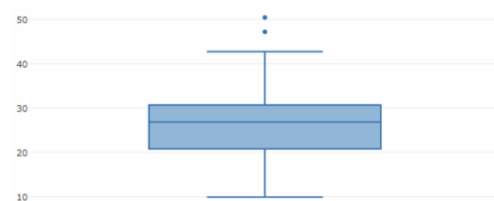


Figure 16: Minification des fichiers CSS (Boîte à moustache)

Nous voyons qu'il n'y a seulement que 2 valeurs aberrantes par type de fichiers et que les valeurs se situent proche de la médiane. Lorsque nous regardons les courbes de la taille minimisée en fonction de la taille initiale, nous pouvons voir une tendance se créer :

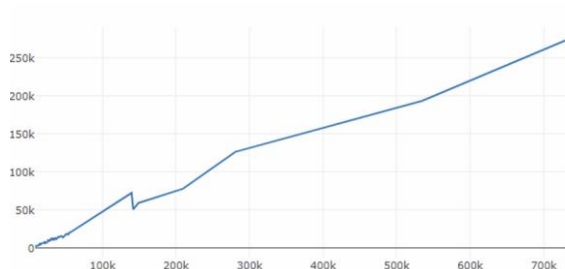


Figure 17: Courbe de tendance des fichiers JavaScript

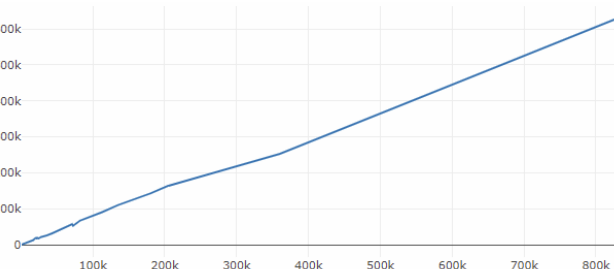


Figure 18: Courbe de tendance des fichiers CSS

Comme une tendance existe pour chaque type de fichiers, le calcul du gain moyen pour chacun est :

- 65% d'octets gagnés pour un fichier JavaScript
- 27% d'octets gagnés pour un fichier CSS.

3.2. Exclure les composants inutilisés des bibliothèques .js

L'exclusion de composants inutilisés des bibliothèques JavaScript consiste à ne pas inclure, dans les scripts téléchargés, des parties du script qui ne seront pas utilisés dans le site internet. Malgré toutes les recherches effectuées sur le sujet, il m'est impossible de trouver une façon correcte de l'implémenter. Toutes les librairies ne proposent pas cette solution d'exclusion, ce qui rend impossible le calcul d'une règle générale. De plus, il est impossible de savoir à l'avance quel sont les composants qui ne seront pas utilisés car ils possèdent chacun une utilité bien précise et une taille différente. Cette inconnue ne permet pas de calculer en elle-même l'espace gagné de manière consistante.

3.3. Chargement asynchrone et différé pour les fichiers CSS et JavaScript.

Le chargement asynchrone de fichiers permet de télécharger un fichier précis sans bloquer le rendu d'une page ou le téléchargement d'un autre fichier. Pour tester cette solution, 50 scripts différents ont été téléchargés, une première fois de façon synchrone, puis une seconde fois de façon asynchrone. Pour être le plus précis dans l'expérience, ce processus a été répété 50 fois :

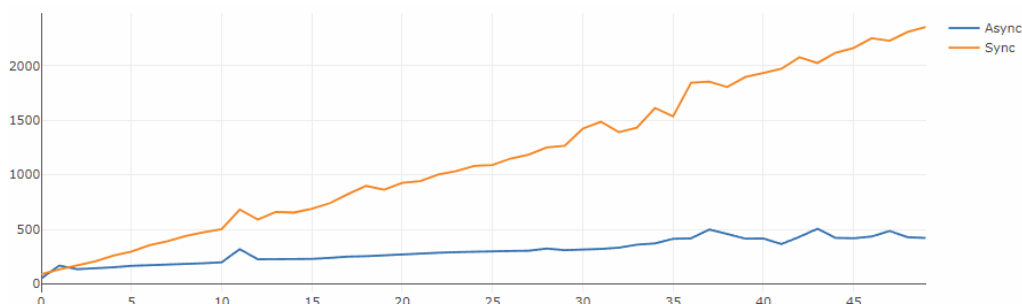


Figure 19: Comparaison du temps de téléchargements de fichiers JavaScript et CSS de façon asynchrone et synchrone

Nous pouvons voir sur nos résultats que, plus de fichiers sont téléchargés, plus le temps de téléchargement est moindre. Pour obtenir et voir si la règle est généralisable, nous allons comparer le pourcentage de temps gagnés, par nombre de fichier, si l'on utilisait le téléchargement asynchrone :

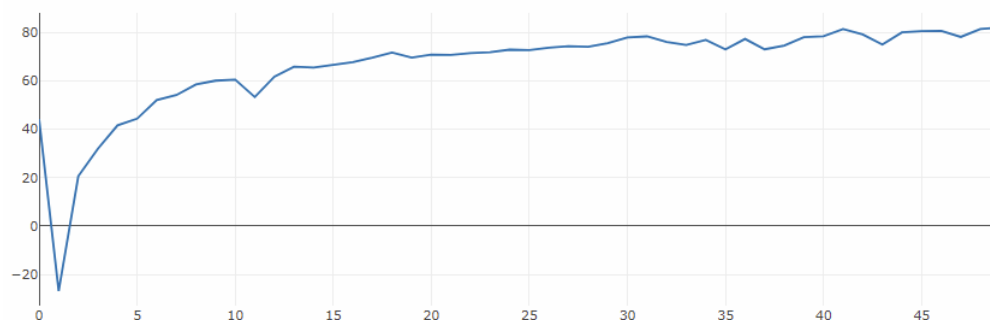


Figure 20: Comparaison du pourcentage de temps gagnés si l'on utilise un chargement asynchrone

Les 3 premiers résultats seront exclus de notre généralisation car ils ne sont pas exploitables. Le chargement asynchrone est une fois meilleur, une fois pire. Cette erreur est explicable par l'utilisation de ressources supplémentaires par la mise en place de la solution asynchrone. Cependant, pour le téléchargement de plus de 3 scripts en même temps, l'utilisation de requêtes asynchrones permet de gagner, en moyenne, 60% de temps de chargement.

3.4. Visibilité de contenu

Lors du chargement d'une page, le contenu peut être « caché » (non chargé sur le page) pour gagner en performance. Pour tester ce gain de performance, 50 images différentes vont être chargées depuis google. Une page web possédera l'attribut de visibilité de contenu et l'autre non (elle sera chargée de façon classique). Pour obtenir nos résultats, comme nous ne modifions pas la taille des fichiers, nous comparerons le temps d'affichage des différentes images pour voir si l'optimisation est efficace :

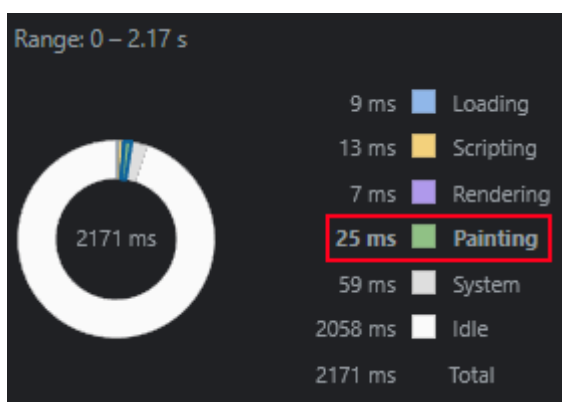


Figure 21: Chargement d'image SANS visibilité de contenu

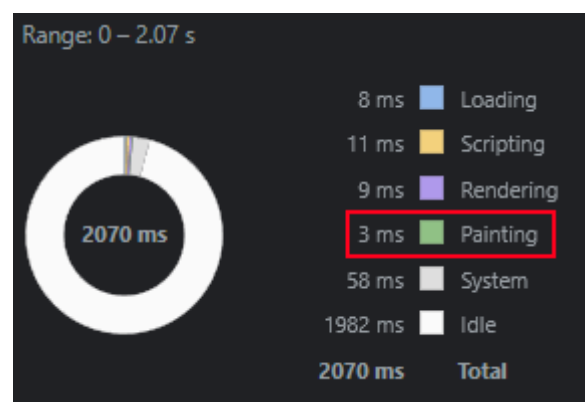


Figure 22: Chargement d'image AVEC visibilité de contenu

Les résultats sont plutôt surprenants. Les 50 images ne mettent en réalité que 25 millisecondes à être affichées sur un ordinateur. Enlever ce temps de rendu permet de le réduire à 3 millisecondes. La différence est tellement dérisoire que cette solution est invalide car l'optimisation est trop peu efficace.

3.5. Optimisation des images

L'optimisation des images consiste à réduire, compresser ou changer le format d'une image. Pour tester cette solution, nous allons prendre 50 photos différentes et les changer de format. Le premier format sera un format PNG, le second JPG et le dernier WEBP. Ce sont les trois différents formats disponibles pour une image. Ensuite, pour chaque image, nous allons comparer leurs tailles et voir le pourcentage d'octets gagnés :

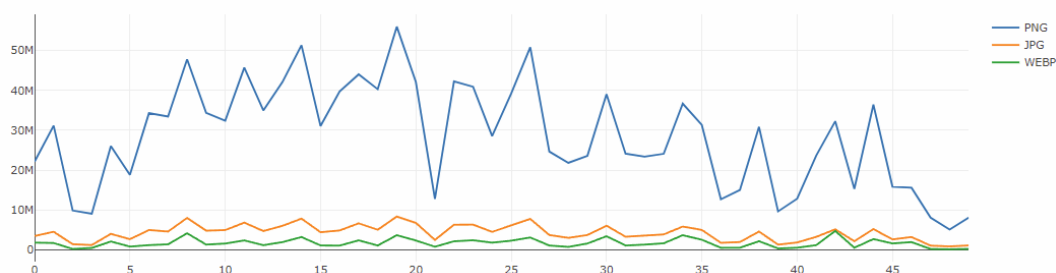


Figure 23: Différence de taille d'une même image en fonction de son format

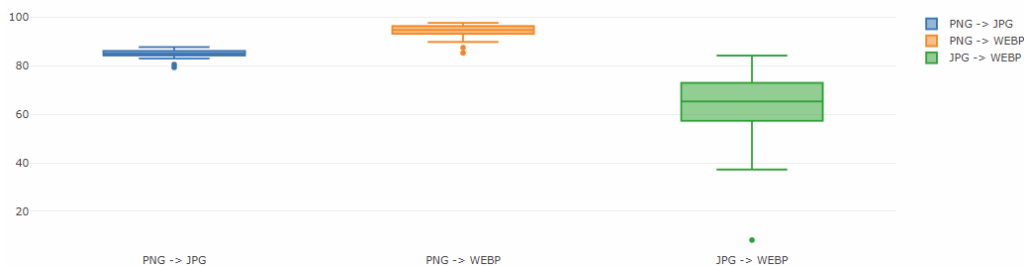


Figure 24: Pourcentage de place gagné par optimisation

Nous pouvons voir qu’une règle générale peut être trouvée car il n’existe que très peu de valeur aberrante. Après analyse des résultats de compression, nous obtenons :

- 85.08% de gains de place pour une compression de PNG vers le format JPG
- 63.11% de gains de place pour une compression de JPG vers le format WEBP
- 94.39% de gains de place pour une compression de PNG vers le format WEBP

3.6. Chargement des images “lazy”

Le chargement lazy d’une image permet de ne pas charger les images non visibles. Après de simple vérification, il n’est pas nécessaire de faire une expérience pour trouver le résultat attendu. En effet, comme aucune image n’est chargée si elle n’est pas visible par l’utilisateur, cela signifie que nous gagnons 100% de temps de chargement pour toutes les images qui ne sont pas affichées à l’écran.

3.7. Décodage asynchrone des images

Le décodage est l’opération consistant à élaborer des octets d’informations pour colorer correctement les pixels. Pour vérifier l’efficacité de la solution, nous allons charger une page internet avec 50 images décodées de façon asynchrone et une page internet avec 50 images sans décodage asynchrone. La moyenne du temps de chargement, sur 100 occurrences, nous donne :

- Temps moyen de chargement pour le décodage asynchrone : 380 secondes
- Temps moyen de chargement pour le décodage synchrone : 378 secondes

La différence du temps de chargement pour une image revient à 0.04 secondes. Ce résultat est trop petit pour être intéressant dans notre étude.

3.8. L’attribut “srcset”

L’attribut srcset permet de charger une image d’une taille différente en fonction de la taille de l’écran de l’utilisateur. Pour tester si cette solution est valide, nous disposons de 50 photos différentes déclinées en 3 tailles différentes. Nous testons chaque taille d’écran 100 fois pour obtenir des résultats cohérents. Les résultats obtenus après ces multiples tests sont :

- Temps de chargement moyen pour les grandes images : 104.08 secondes
- Temps de chargement moyen pour les moyennes images : 70.53 secondes
- Temps de chargement moyen pour les petites images : 54.01 secondes

Pour les grands écrans, aucun gain n’est présent. Cependant, pour les écrans moyens et petits, nous gagnons en temps de chargement. En moyenne, une image déclinée sur une taille moyenne permet de gagner 0.571 secondes par images. Le temps pour une image déclinée en petite taille, elle, permet de gagner sur un écran de téléphone jusqu’à 1 seconde par image !

Conclusion

Parmi les différents résultats obtenus, nous pouvons les mettre dans un tableau pour nous aider à les trier. En plus d'en sortir une règle générale de temps gagné, nous allons y ajouter la difficulté de la mise en place de cette solution pour les départager :

Nom de l'optimisation	Gain	Difficultés de mise en place
Minification des fichiers JavaScript	En moyenne 65%	Facile
Minification des fichiers CSS	En moyenne 27%	Facile
Exclusion des composants inutilisés des bibliothèques JS		
Chargements asynchrones pour les fichiers externes	Pour plus de 3 fichiers, en moyenne 65%	Facile
Visibilité de contenu		
Optimisation des images au format PNG	En moyenne 94.39%	Moyen
Optimisation des images au format JPG	En moyenne 63.11%	Moyen
Chargement des images « lazy »	Pour les images non visible, 100%	Facile
Décodage asynchrone des images		
Attribut srcset pour des écrans moyen	En moyenne 0.571 seconde	Difficile
Attribut srcset pour des petits écrans	En moyenne 1 seconde	Difficile

Nous pouvons voir que sur nos 8 expériences, nous pouvons sortir 8 règles générales ! Cependant 3 des expériences ne nous mènent à rien. Ces expériences sont un échec dû à l'impossibilité de les mettre en place pour les tester où sont soumis à des facteurs trop aléatoire qui fausserai totalement les résultats. Ce tableau nous permet de les trier par gain de temps et par difficulté pour obtenir notre liste finale qui contient, dans un ordre croissant, les optimisations à prioriser par n'importe qui. Cette liste permet de se focaliser sur les optimisations les plus pertinentes et les meilleurs pour minimiser le coût de l'optimisation d'un site internet :

Nom de l'optimisation	Gain	Difficultés de mise en place
Chargement des images « lazy »	Pour les images non visible, 100%	Facile
Minification des fichiers JavaScript	En moyenne 65%	Facile
Chargements asynchrones pour les fichiers externes	Pour plus de 3 fichiers, en moyenne 65%	Facile
Minification des fichiers CSS	En moyenne 27%	Facile
Optimisation des images au format PNG	En moyenne 94.39%	Moyen
Optimisation des images au format JPG	En moyenne 63.11%	Moyen
Attribut srcset pour des écrans moyen	En moyenne 0.571 seconde	Difficile
Attribut srcset pour des petits écrans	En moyenne 1 seconde	Difficile

Lors de cette étude, nous avons réalisé 8 expériences différentes et nous avons réussie à définir 8 règles de gain de temps par optimisation. Cette étude est donc réussie à la vue des objectifs fixés au début de celle-ci. Bien que cette étude soit complète sur l'optimisation d'un site Internet, elle ne couvre qu'un seul domaine bien particulier. Il est possible et totalement imaginable d'étendre cette étude pour les optimisations de connexion internet (réseaux au niveau des serveurs WEB) ou encore les optimisations des serveurs WEB en eux-mêmes (gestion du cache, etc...).

Bibliographie

- REEBS S. **La méthode scientifique**. 2017. [En ligne]. Université de Moncton, Canada. Disponible sur <https://www.umoncton.ca/umcm-sciences/files/sciences/wf/wf/pdf/la_methode_scientifique.pdf>. Consulté le 15/12/2021.
- LETERME C. **Style de rédaction scientifique : conseils, méthodologie, exemples**. 10 février 2020. [En ligne]. Scribbr. Disponible sur <<https://www.scribbr.fr/article-scientifique/style-redaction-scientifique/>>. Consulté le 15/12/2021.
- LETERME C. **La démarche scientifique : tout ce que vous devez savoir !** 18 décembre 2019. [En ligne]. Scribbr. Disponible sur <<https://www.scribbr.fr/article-scientifique/demarche-scientifique/>>. Consulté le 15/12/2021.
- **Taux de rebond**. 30 juin 2020. [En ligne]. Wikipédia. Disponible sur <https://fr.wikipedia.org/wiki/Taux_de_rebond>. Consulté le 15/12/2021.
- **Définition Taux de rebond**. [En ligne]. SEO.fr. Disponible sur <<https://www.seo.fr/definition/taux-de-rebond>>. Consulté le 16/12/2021.
- TERENTAVA E. **Website Performance Research**. 8 mai 2018. [En ligne]. Semrush Blog. Disponible sur <<https://www.semrush.com/blog/website-performance-semrush-study/>>. Consulté le 16/12/2021.
- **30+ Website Load Time Statistics**. 2021. [En ligne]. WebsiteSetup. Disponible sur <<https://websitesetup.org/news/website-load-time-statistics/>>. Consulté le 10/01/2022.
- MONAGHAN M. **Website Load Time Statistics: Why Speed Matters in 2022**. 14 janvier 2022. [En ligne]. WebsiteBuilderExpert. Disponible sur <<https://www.websitebuilderexpert.com/building-websites/website-load-time-statistics/>>. Consulté le 10/01/2022.
- **Find Out How You Stack Up to New Industry Benchmarks for Mobile Page Speed**. Février 2017. [En ligne]. Google. Disponible sur <<https://think.storage.googleapis.com/docs/mobile-page-speed-new-industry-benchmarks.pdf>>. Consulté le 15/01/2022.
- DEAN B. **Here's What We Learned About Page Speed**. 8 octobre 2019. [En ligne]. Backlinko. Disponible sur <<https://backlinko.com/page-speed-stats>>. Consulté le 22/01/2022.
- **Mobile share of total digital minutes in leading online markets as of August 2020**. Novembre 2020. [En ligne]. Statista. Disponible sur <<https://www.statista.com/statistics/266379/mobile-share-of-digital-minutes-countries/>>. Consulté le 12/02/2022.
- **Think Fast, The Page Speed Report Stats & Trends for Marketers**. [En ligne]. Unbounce. Disponible sur <<https://unbounce.com/page-speed-report/>>. Consulté le 14/02/2022.
- **Milliseconds make Millions**. 2020. [En ligne]. Deloitte. Disponible sur <https://www2.deloitte.com/content/dam/Deloitte/ie/Documents/Consulting/Milliseconds_Make_Millions_report.pdf>. Consulté le 14/02/2022.
- GALOV N. **21+ Website Load Time Statistics and Facts**. [En ligne]. Hosting Tribunal. Disponible sur <<https://hostingtribunal.com/blog/how-speed-affects-website/#gref>>. Consulté le 14/02/2022.
- SMITH C. **Page Speed and Decreased Conversion Rates: 2019 Statistics**. [En ligne]. Trinity. Disponible sur <<https://trinity.one/insights/user-experience/page-speed-conversion-2019-statistics/>> Consulté le 15/02/2022.
- TODOROV G. **80+ Relevant Site Speed Statistics and Trends in 2022**. 14 mars 2022. [En ligne]. ThriveMyWay. Disponible sur <<https://thrivemyway.com/site-speed-stats/>>. Consulté le 18/02/2022.
- DIGGA K. **How the Web works**. 18 février 2022. [En ligne]. Mozilla. Disponible sur <<https://developer.mozilla.org/en->

[US/docs/Learn/Getting_started_with_the_web/How_the_Web_works](#)>. Consulté le 18/02/2022.

- KASIREDDYP. **How the Web Works: A Primer for Newcomers to Web Development (or anyone, really)**. [En ligne]. Freecodecamp. Disponible sur <https://www.freecodecamp.org/news/how-the-web-works-a-primer-for-newcomers-to-web-development-or-anyone-really-b4584e63585c/>>. Consulté le 10/03/2022.
- Academind. **How The Web Works**. 15 avril 2019. [En ligne]. Youtube. Disponible sur <https://www.youtube.com/watch?v=hJHvdBISxug>>. Consulté le 10/03/2022.
- Dj3. **What happens when you load a webpage?**. 11 avril 2022. [En ligne]. Youtube. Disponible sur <https://www.youtube.com/watch?v=vJvhxhpa6I>>. Consulté le 10/03/2022.
- ZIMMERMAN A. **How a web browser builds and displays a web page**. 12 octobre 2013. [En ligne]. Youtube. Disponible sur <https://www.youtube.com/watch?v=DuSURHrZG6I>>. Consulté le 13/03/2022.
- Code Sensei. **How a web application works**. 3 mai 2020. [En ligne]. Youtube. Disponible sur <https://www.youtube.com/watch?v=zjFViRCJT5U>>. Consulté le 14/03/2022.
- SHIOTSU Y. **JavaScript Optimization Tips To Improve Performance in 2022**. 16 août 2021. [En ligne]. Upwork. Disponible sur <https://www.upwork.com/resources/javascript-optimization-tips>>. Consulté le 14/03/2022.
- PIAZZOLLA G. **IMG TAG Loading Optimization**. 2 mai 2021. [En ligne]. Medium CodeX. Disponible sur <https://medium.com/codex/the-img-tag-performance-optimization-using-standard-html-attributes-8403c14d0de9>>. Consulté le 16/03/2022.
- DUO M. **9 Quick Ways to Improve Page Loading Speed**. 25 mai 2021. [En ligne]. Hubspot. Disponible sur <https://blog.hubspot.com/marketing/how-to-reduce-your-websites-page-speed>>. Consulté le 18/03/2022.
- **12 Techniques of Website Speed Optimization: Performance Testing and Improvement Practices**. 30 avril 2018. [En ligne]. Altexsoft. Disponible sur <https://www.altexsoft.com/blog/engineering/12-techniques-of-website-speed-optimization-performance-testing-and-improvement-practices/>>. Consulté le 20/03/2022.
- OSMANI A. **JavaScript Start-up Optimization**. 2 juillet 2018. [En ligne]. Web.dev. Disponible sur <https://web.dev/optimizing-content-efficiency-javascript-startup-optimization/>>. Consulté le 01/04/2022.
- GÜBÜR K. **Optimiser les ressources JavaScript pour gagner en vitesse**. 5 juin 2020. [En ligne]. Oncrawl. Disponible sur <https://fr.oncrawl.com/seo-technique/optimiser-les-ressources-javascript-pour-gagner-en-vitesse/>>. Consulté le 01/04/2022.
- RANKIN T. **How to Optimize JavaScript and CSS and Improve Website Performance**. 27 juin 2019. [En ligne]. Torquemag. Disponible sur <https://torquemag.io/2019/06/optimize-javascript-css/>>. Consulté le 01/04/2022.
- RAVENSBERGEN K. **How to Optimize JavaScript Delivery to Speed Up Your Site**. [En ligne]. Blue Triangle. Disponible sur <https://blog.bluetriangle.com/js-delivery-optimization-for-web-performance>> Consulté le 05/04/2022.
- **A Beginner's Guide to Website Speed Optimization**. 21 mars 2022. [En ligne]. Kinsta. Disponible sur <https://kinsta.com/learn/page-speed/>>. Consulté le 10/04/2022.

Glossaire

- **Website** : Un site web est un ensemble de pages web qui peuvent être consultées en suivant des hyperliens à l'intérieur du site.
- **CSS** : Acronyme de « Cascading Style Sheets » ce qui signifie « feuille de style en cascade ». Ils permettent de styliser une page internet.
- **JavaScript** : Langage de développement informatique, et plus précisément un langage de script orienté objet.
- **.js** : Extension de fichier associé à des scripts JavaScript.
- **DOM** : Le Document Object Model (DOM) est une interface de programmation normalisée par le W3C, qui permet à des scripts d'examiner et de modifier le contenu du navigateur web.

Annexes

L'entièreté des tests réalisés ainsi que les images utilisées lors de ce projet de démarche d'application à la méthode scientifique est consultable sur GitHub : <https://github.com/ClementGaston/ADS>.